Biashara Bridges — Project Architecture

This document captures a professional-level architecture analysis for the Django project in this repository. It describes the current structure, responsibilities, strengths, weaknesses, identified gaps to reach "world-class" maturity, and prioritized remediation recommendations with suggested next steps.

Location: `docs/ARCHITECTURE.md`

---

1. Executive Summary

This repository is a Django-based monolith that serves the Biashara Bridges website (landing page + additional apps). It includes multiple Django apps (`accounts`, `marketplace`, `payments`), a `config` package for Django settings, templates and static assets, service modules for payment integrations, a sqlite DB (for development), and helper scripts.

Overall maturity: medium. The codebase is functional and pragmatic: clear apps, services grouped under `payments/services`, and templates for the single-page landing site. To reach professional / world-class standards, the project requires improvements across configuration management, packaging, deployment, observability, testing, security, and modular design.

---

2. High-Level Architecture

- Platform: Django (WSGI/ASGI capable, `config` package contains `settings.py`, `urls.py`, `wsgi.py`, `asgi.py`).
- Apps (domain partitions):
- `accounts` — authentication, user profiles, custom backends and forms
- `marketplace` — marketplace models, views, templates
- `payments` — payment models, signals, webhooks, services (stripe/paypal/mpesa), `services/` holds adapters
- Templates: server-side Django templates (`templates/` root) and numerous Element-orientated HTML artifacts (landing page). Landing page heavily references static assets.
- Static: `static/` contains CSS/JS/fonts; `static/landing/` holds downloaded/replicated assets for the landing page.
- DB: SQLite present in repo (`db.sqlite3`) — used for rapid dev/test only.
- Scripts: `scripts/*.py` utility scripts for generating data and fetching assets.
- Integrations: third-party payment providers (Stripe, PayPal, Mpesa), webhooks, email (likely), external asset references.

Design pattern highlights:
- Services/adapters pattern inside `payments/services` is a good separation of concerns for payment integrations.
- Signals used in `payments/signals.py` for side effects.

---

3. File & Package Map (important files only)

- `manage.py` — Django management entry.
- `config/` — Django config (`settings.py`, `urls.py`, `wsgi.py`, `asgi.py`).
- `accounts/` — app for users/authorization.
- `marketplace/` — business logic models/views/templates.
- `payments/` — payments app, webhooks, services, signals.
- `templates/` — many site templates incl. `landing.html`.
- `static/` — local static assets.
- `db.sqlite3` — local DB (should not be used in production repo).
- `requirements.txt` — dependencies (pinning needs verification).
- `scripts/` — small utilities used for local asset fetching and data creation.

---

4. Strengths (what is already good)

- Logical app separation: payments, accounts, marketplace are separated into Django apps.
- Services folder for payment adapters demonstrates some clean architecture thinking (adapter/strategy pattern).
- Use of Django migrations under each app (migrations present). This is good for schema management.
- Templates & static separation follow Django conventions.
- Presence of tests files in apps (`tests.py`) — shows intent to test (coverage level TBD).
- Helper scripts exist for repeatable local tasks (data creation, asset fetching).

---

5. Key Weaknesses, Pain Points & Gaps (analysis)

Below are concrete pain points preventing this project from being "world-class".

1. Configuration & Secrets Management
- `config/settings.py` likely centralizes environment and secrets. No clear separation of `development` / `staging` / `production` settings or use of environment variables. Secrets may be stored in source or implicitly present via default settings.
- No documented environment file pattern (e.g., `django-environ`, `.env`) nor a secrets manager integration.

2. Deployment / Packaging
- No `Dockerfile` or containerization artifacts present (no `docker-compose.yml`).
- No CI/CD pipeline or automation for tests, linting, build, or deployment (no `.github/workflows` present).
- `db.sqlite3` checked into repository — not appropriate for production and risks leaking data.

3. Static/media management
- Large number of static assets are stored in the repo (`static/landing`) — fine for local dev but not ideal for production. No `collectstatic` strategy for S3/Cloud storage or CDN usage.
- No asset build pipeline (no webpack, esbuild, or similar) and no clear cache-busting strategy.

4. Observability & Error Monitoring
- No centralized structured logging, no Sentry/exception monitoring integration, and no healthchecks.

5. Asynchronous processing & scalability
- Payments and webhook handling are synchronous. No background job queue for long-running tasks (e.g., Celery with Redis) — webhooks and payment reconciliation may block requests.

6. Security
- No automated checks for dependencies (vulnerability scanning) or dependency pinning policies validated.
- No security hardening guidance (e.g., SSL, HSTS, secure cookies, CSP) visible in settings.

7. Testing & CI
- Tests exist but scope and coverage are unknown. No CI automation to run tests, linters, or style checks on PRs.

8. Code Hygiene & Quality
- Likely missing type annotations (mypy), linter configs (`flake8`/`ruff`) and pre-commit hooks.

- Potential duplication between services and views; no clear domain layer separation (business/domain logic probably mixed into views/models).

9. Documentation & Onboarding
- No central architecture or developer onboarding docs (this is being added). No `README` with quick-start, no `CONTRIBUTING.md`, no dev environment instructions (venv, dependencies, how to run local server securely).

10. Data & Migrations Practice
- Single `db.sqlite3` in repo may defeat testing/reproducibility and can hide migration drift. Not necessarily a problem for small projects, but for production it's a risk.

11. API & Contract Stability
- For external integrations (payments), there is likely no versioned public API or clear contract tests for webhook payloads.

---

6. Recommendations (prioritized)

Below are recommended actions, ordered by priority for reducing risk and moving toward world-class standards.

Top priority (immediate — 1–2 weeks):

1. Secrets & Settings Hardening
- Extract all secrets and environment-specific settings from `config/settings.py` into environment variables using `django-environ` or equivalent.
- Create `config/settings/` package with `base.py`, `development.py`, `production.py`, `local.py`. Each file imports from `base.py` and overrides env-specific values.
- Add a `.env.example` (no secrets) and document required env vars in README.

2. Remove `db.sqlite3` from repo & Add dev-only option
- Remove `db.sqlite3` from source control (add to `.gitignore`). If you need a dev DB, provide a script to create a throwaway sqlite or use `docker-compose` to bring up a Postgres dev DB.

3. Implement a minimal CI pipeline
- Add `.github/workflows/ci.yml` to run tests, flake/ruff linting, and safety checks on PRs. This prevents regressions and ensures basic hygiene.

4. Add dependency pinning + vulnerability checks
- Switch to pinned `requirements.txt` or use `pip-tools` (`requirements.in` -> `requirements.txt`) or `poetry` for reproducible installs.
- Add `safety` or GitHub Dependabot for automated dependency scanning.

Medium priority (2–6 weeks):

5. Containerize & provide reproducible local environment
- Add `Dockerfile` (python slim + gunicorn) and `docker-compose.yml` for local dev (Postgres, Redis for Celery). This standardizes dev and deployment.

6. Introduce background processing for payments
- Add Celery + Redis for asynchronous processing (webhook processing, reconciliation tasks). Convert blocking tasks in webhooks/signals to background jobs.

7. Static & Media strategy
- For production, use `django-storages` with S3/Cloud storage and serve static assets via CDN. Use `collectstatic` in CI/CD.
- Add an asset build step (optional) for JS/CSS bundling and optimization.

8. Observability & Error Tracking
- Integrate Sentry (or equivalent) for exception monitoring and performance traces.

- Standardize logging (structured JSON output) and expose a `/health` endpoint for monitoring.

Longer-term / formalization (6–12 weeks):

9. Improved testing & contract tests
- Expand unit and integration tests, add contract tests for payment webhooks.
- Add end-to-end smoke tests for key flows (e.g., sign-up, payments). Use GitHub Actions matrix to run tests across Python versions.

10. Code quality & developer experience
- Add pre-commit hooks (black/ruff/isort), type checking (mypy) and static code analysis.
- Add `CONTRIBUTING.md`, `README.md` with dev setup and local run instructions.

11. Architecture hardening
- Consider refactoring toward a modular/domain-driven layout: separate `services/` or `domain/` layer from web layer. Introduce well-documented interfaces between adapters (payments) and domain logic.
- For high-scale needs, evaluate breaking off critical components (payments/reconciliation, background processing) into separate services.

---

7. Concrete Implementation Steps (recommended quick wins)

1. Create `docs/` (done) and add this `ARCHITECTURE.md`.
2. Add `.env.example`, document `DJANGO_SECRET_KEY`, DB vars, `SENTRY_DSN`, `REDIS_URL`, `STRIPE_*`, `PAYPAL_*`, `MPESA_*`. Use `django-environ`.
3. Remove `db.sqlite3` and add `scripts/setup_dev_db.py` that can create a clear dev DB and seed minimal data.
4. Add GitHub Actions `ci.yml` to run: `pip install -r requirements.txt`, `python -m pytest`, `ruff check`, `mypy` (optional).
5. Add `Dockerfile` + `docker-compose.yml` with Postgres & Redis to reproduce environment locally.
6. Convert long-running webhook processing to Celery tasks; keep webhook endpoint fast and ack quickly.
7. Add Sentry and structured logging.
8. Add `requirements.in` + `pip-compile` to pin transitive dependencies, and enable Dependabot.

---

8. Security checklist (must-haves before production)

- Do NOT commit secrets. Replace any secrets in repo with env-based config.
- Set `DEBUG=False` in production and enable `ALLOWED_HOSTS` correctly.
- Use HTTPS, set `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS`.
- Use a robust password hashing algorithm and enforce password policies.
- Rate-limit public endpoints (especially login & webhook endpoints) and validate webhook signatures.
- Use dependency scanning and periodic license checks.

---

9. Observability & SLOs

- Add Sentry for error reporting, and a Prometheus + Grafana stack or cloud alternative for metrics.
- Add basic SLOs: 99.5% uptime for site availability, 99% success for payments API calls, 95th percentile latency thresholds for page and API responses.

---

10. Example Roadmap (quarterly)

Quarter 1 (foundations): env & secrets, CI, remove sqlite, README, add

`.env.example`.
Quarter 2 (ops & reliability): Docker, static/media strategy, Sentry, logging, healthchecks.
Quarter 3 (scalability): Celery & background jobs, Postgres optimisation, caching (Redis), add tests & contract tests.
Quarter 4 (polish): Type hints, stricter linting, performance optimizations, optional microservice boundary extraction.

---

11. Estimated Effort (very rough)

- Quick wins (CI, `.env`, README, remove sqlite): 2–5 days
- Containerization + Docker Compose + S3 static setup: 5–10 days
- Background worker (Celery) + webhook refactor: 5–10 days
- Tests & CI coverage improvements: 2–6 weeks depending on existing test coverage

---

12. Offer to help

I can:
- Implement immediate quick wins: create `Dockerfile`, `docker-compose.yml`, `ci.yml`, or refactor settings into `config/settings/*.py` and add `.env.example`.
- Implement Celery + Redis scaffolding and convert one webhook to a background task.
- Add GitHub Actions to run tests and linting.

Tell me which priority item you want me to implement first and I will put together a plan and start implementing it.

(End of architecture document)

Immediate High-Impact Fix (Do this first)

The single fastest, highest-impact item that moves the project toward professional/world-class readiness is:

- Extracting all configuration and secrets out of `config/settings.py` into environment variables and reorganizing settings into environment-specific modules (development / production / local).

Why this is highest-impact:
- Prevents accidental secret/key leakage into source control.
- Enables safe CI/CD and multi-environment deployment (staging, production).
- Is a prerequisite for secure production settings (DEBUG=False, ALLOWED_HOSTS, secure cookies) and for containerization.

Clear step-by-step plan to resolve this (comprehensive, maximal)

This expanded plan is written as a practical implementation playbook. Each major step includes sub-steps, commands, verification checks, CI examples, rollbacks, and estimated time.

Major deliverables
- `config/settings/` package with `base.py`, `development.py`, `production.py`, `local.py`.
- `.env.example` and developer docs.
- `.gitignore` updated to exclude runtime secrets and local DB.
- A GitHub Actions workflow `ci/secret-check.yml` + `ci/tests.yml` to verify no secrets and to run tests/lint.
- A documented secret-rotation plan and optional `scripts/rotate_secrets.md` with steps.

Step 1 — Create a settings package (estimated 2–4 hours)

1.1 Create folder `config/settings/` and move `config/settings.py` -> `config/settings/base.py`.
1.2 Create `development.py`, `production.py`, `local.py` that each `from .base import *` then override env-specific bits.
1.3 Update `config/__init__.py` or the `manage.py` / `wsgi.py` / `asgi.py` environment to point `DJANGO_SETTINGS_MODULE` at a selector (e.g. `config.settings.development` for local). Use an environment variable `DJANGO_ENV` to choose automatically in deployment scripts.

Example layout:

```
config/
settings/
__init__.py   # loads settings according to DJANGO_ENV or fallback
base.py
development.py
production.py
local.py
```

Snippet for `config/settings/__init__.py`:

```python
import os
env = os.getenv("DJANGO_ENV", "development")
if env == "production":
from .production import *
elif env == "local":
from .local import *
else:
from .development import *
```

Why and verification
- Why: This enables per-environment configuration and makes production-only settings clearly visible.
- Verify: `python manage.py check --deploy` (after setting env) and ensure no sensitive defaults remain.

Step 2 — Add environment loader & define required variables (estimated 1–2 hours)
2.1 Add `django-environ` to `requirements.txt` (or `python-dotenv` if preferred).
2.2 In `base.py` load environment at the top and declare all required variables and defaults. Use `env.bool()` for flags.

Example snippet for `base.py` using `django-environ`:

```python
import environ
env = environ.Env(
DEBUG=(bool, False),
)
environ.Env.read_env()  # reads .env for local

SECRET_KEY = env("DJANGO_SECRET_KEY")
DEBUG = env.bool("DEBUG", default=False)
DATABASES = {"default": env.db()}
ALLOWED_HOSTS = env.list("ALLOWED_HOSTS", default=["localhost"])
```

2.3 Create a declarative list `REQUIRED_ENV_VARS` and at startup assert they are present in production mode. Example check inside `base.py`:

```python
```

if not DEBUG:
missing = [k for k in REQUIRED_ENV_VARS if not env(k, default=None)]
if missing:
raise RuntimeError("Missing required env vars: %s" % ",".join(missing))
```

Why and verification
- Why: Centralized env loading enforces consistent parsing of DB URLs, booleans, lists and reduces ad-hoc reading of os.environ across the codebase.
- Verify: run `python -c "import django; print('env ok')"` after setting minimal env vars.

Step 3 — Provide `.env.example` and update docs (estimated 30–60 minutes)
3.1 Add `.env.example` at repo root with annotated keys (no secrets). Suggested keys:

```
# Django
DJANGO_ENV=development
DJANGO_SECRET_KEY=
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost

# Database
DATABASE_URL=sqlite:///./db.sqlite3

# Sentry / Observability
SENTRY_DSN=

# Redis / Celery
REDIS_URL=redis://localhost:6379/0

# Payment providers
STRIPE_SECRET=
PAYPAL_CLIENT_ID=
PAYPAL_SECRET=
MPESA_KEY=
```

3.2 Document in `README.md` the exact commands to create `.env` from `.env.example` and how to run the dev server.

Why and verification
- Why: Ensures new contributors can bootstrap quickly and consistently.
- Verify: Create `.env` locally from `.env.example` and run `python manage.py runserver`.

Step 4 — Prevent secrets from being committed & clean history if required (estimated 1–3 hours)
4.1 Update `.gitignore` to include:

```
.env
db.sqlite3
*.py[cod]
__pycache__/
```

4.2 If secrets already exist in git history:
- Identify sensitive commits with `git log -S "SECRET_KEY"` or use `git-secrets` to scan.
- Rotate any secrets exposed (API keys, DB passwords) immediately with their providers.
- If necessary, rewrite history with `git filter-repo` or `bfg` and coordinate the team to re-clone. Document the steps and perform during a maintenance window.
```

Why and verification
- Why: Adds a safety net preventing future accidental commits and reduces the impact window of leaked credentials.
- Verify: `git ls-files --others --exclude-standard` should not show `.env` or `db.sqlite3`.

Step 5 — Add CI checks to catch regressions (estimated 2–6 hours)
5.1 Add GitHub Actions workflow `ci/secret-check.yml` that:
- Runs a tiny Python script or `grep` to assert that production settings file(s) contain no literal `SECRET_KEY =` assignments or hard-coded tokens.
- Runs `pytest -q` and `ruff` lint.

Example `ci/secret-check.yml` skeleton:

```yaml
name: CI
on: [push, pull_request]
jobs:
checks:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Set up Python
uses: actions/setup-python@v4
with:
python-version: '3.11'
- name: Install deps
run: |
python -m pip install --upgrade pip
pip install -r requirements.txt
- name: Secret scan (simple)
run: |
if grep -R "SECRET_KEY\s*=\s*['\"]" config | grep -v "__init__"; then echo "Found SECRET_KEY in repo"; exit 1; fi
- name: Run tests
run: pytest -q
- name: Lint
run: ruff check .
```

5.2 Optionally add `git-secrets` or use a GitHub Action for secret scanning (e.g., `gitleaks-action`).

Why and verification
- Why: CI gates minimize the chance of secret or config regressions reaching main and ensure tests/linting run on every PR.
- Verify: Create a test commit that intentionally includes a `SECRET_KEY` in a non-excluded file and ensure CI fails.

Step 6 — Verification checklist & acceptance criteria (automated + manual) (estimated 1–2 hours)
- `config/settings/production.py` contains zero hard-coded credentials.
- `.env.example` present at repo root and `.env` does not exist in repo.
- `.gitignore` excludes `.env` and `db.sqlite3` and no secrets appear in changed files in PRs.
- CI workflow runs on PRs and fails if a `SECRET_KEY` literal is present.
- Dev run: contributor can follow README, create `.env` from `.env.example`, and start `manage.py runserver` without secret errors.

Automated checks to add (recommended):
- `pytest` for unit tests.
- `ruff` for linting.
- `gitleaks` or `gitleaks-action` for secret scanning.

Step 7 — Secret rotation & incident response (must-have if secrets were committed)
7.1 Immediately rotate all compromised keys in respective provider consoles (Stripe, PayPal, hosting, Sentry).
7.2 Revoke old keys and issue new ones, update `.env` for deployments.
7.3 If a secret was in git history, run `git filter-repo --path-glob "*" --replace-refs delete-no-add` or BFG as documented; coordinate with the team to re-clone and reset CI tokens.
7.4 Document the event in `SECURITY.md` and notify stakeholders.

Step 8 — Rollout & rollback plan (minimal disruption)
8.1 Implement changes in a feature branch; open a PR and request at least one reviewer.
8.2 Merge after CI passes and after manual verification on a staging environment.
8.3 Rollback: if production shows issues, revert the deployment and reopen the PR, or restore old environment variables from secret manager.

Step 9 — Timeline & responsibility
- Phase A (1–3 days): Implement settings package, `.env.example`, update `.gitignore`, add minimal README snippets.
- Phase B (2–5 days): Add CI workflows, linting, run tests, add simple secret scanning action.
- Phase C (2–7 days): If required, rotate leaked secrets and clean git history; integrate secret manager (HashiCorp Vault / AWS Secrets Manager) for production.

Step 10 — Follow-ups (post completion)
- Containerize (Docker + `docker-compose`) and switch development to Postgres + Redis for realistic parity with production.
- Add Sentry and structured logging for observability.
- Convert webhook handling to background tasks (Celery + Redis) and add contract tests for payment webhooks.

Risk & mitigation summary
- Risk: Rewriting git history when removing secrets is destructive. Mitigation: perform during a scheduled maintenance window, rotate secrets, and communicate the change.
- Risk: CI false positives. Mitigation: start with conservative checks (grep for `SECRET_KEY`) and iterate.

Acceptance criteria (final)
- No secrets in `config` files or committed history (or otherwise rotated and removed from history).
- Clear developer onboarding: `.env.example`, README updated, and quick start commands working.
- CI enforces the rules on PRs and merges.

Ready-to-run commands (PowerShell)

```powershell
# create local env
copy .env.example .env
# install deps
venv\Scripts\Activate; pip install -r requirements.txt
# run locally
python manage.py migrate
python manage.py runserver 127.0.0.1:8000
```

This completes the comprehensive immediate-fix playbook. Execute the steps in small commits and validate each automated check before moving to the next.

(End of added immediate-fix plan)